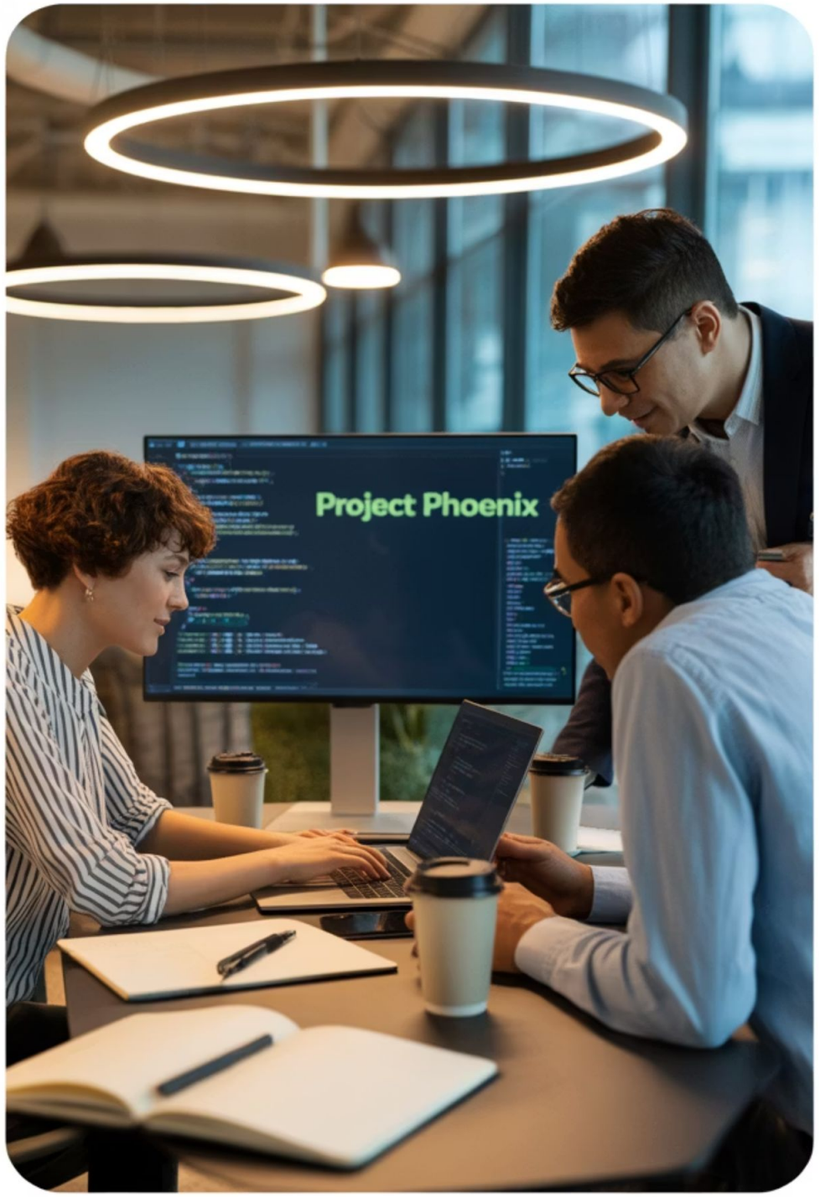
Introducción a UML:

Unified Modeling Language

El lenguaje universal para el diseño de software profesional



¿Por qué modelar antes de programar?

Claridad

Reduce la ambigüedad en los requisitos y permite una comprensión compartida del sistema antes de escribir la primera línea de código.

Comunicación

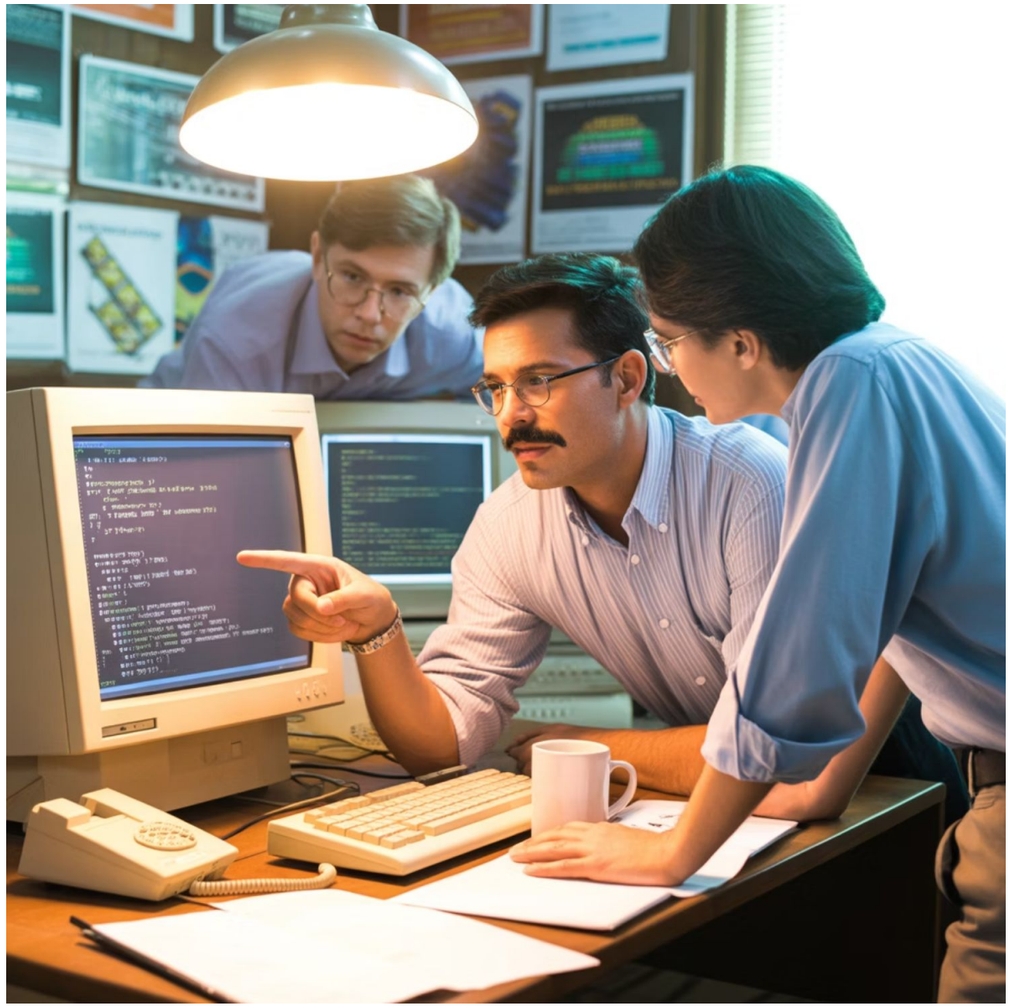
Facilita el diálogo entre desarrolladores, clientes y stakeholders usando un lenguaje visual común y comprensible para todos.

Detección de riesgos

Identifica problemas potenciales en la arquitectura y el diseño antes de invertir tiempo y recursos en la implementación.

Recuerda: **UML es una guía** para el desarrollo, **no el código final**. Su valor está en la planificación y comunicación.

Origen e Historia de UML



Años 90

1

Crisis en métodos de desarrollo: cada empresa usaba su propia notación, generando confusión y falta de estándares en la industria.

"Three Amigos"

2

Grady Booch, James Rumbaugh e Ivar Jacobson unieron sus metodologías para crear un estándar unificado de modelado.

OMG 1997

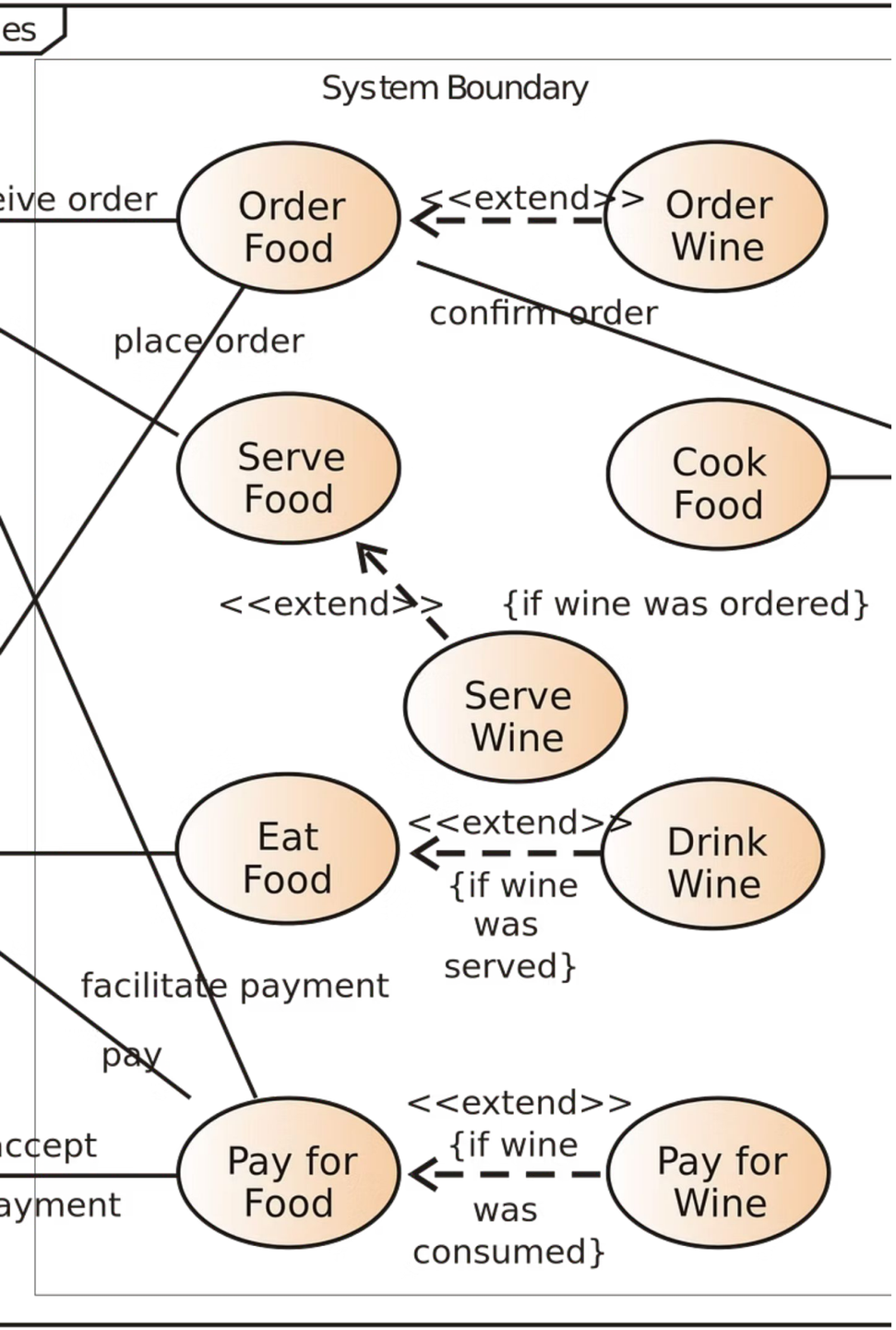
3

Object Management Group adopta UML como estándar oficial, garantizando su evolución y mantenimiento continuo.

UML 2.x

4

Versión actual con 14 tipos de diagramas, ampliamente adoptada en la industria del desarrollo de software mundial.



¿Qué es UML exactamente?

Lenguaje gráfico de modelado

Sistema de notación visual estandarizado que permite representar la estructura y comportamiento de sistemas software de manera intuitiva y profesional.

Independiente de lenguaje de programación

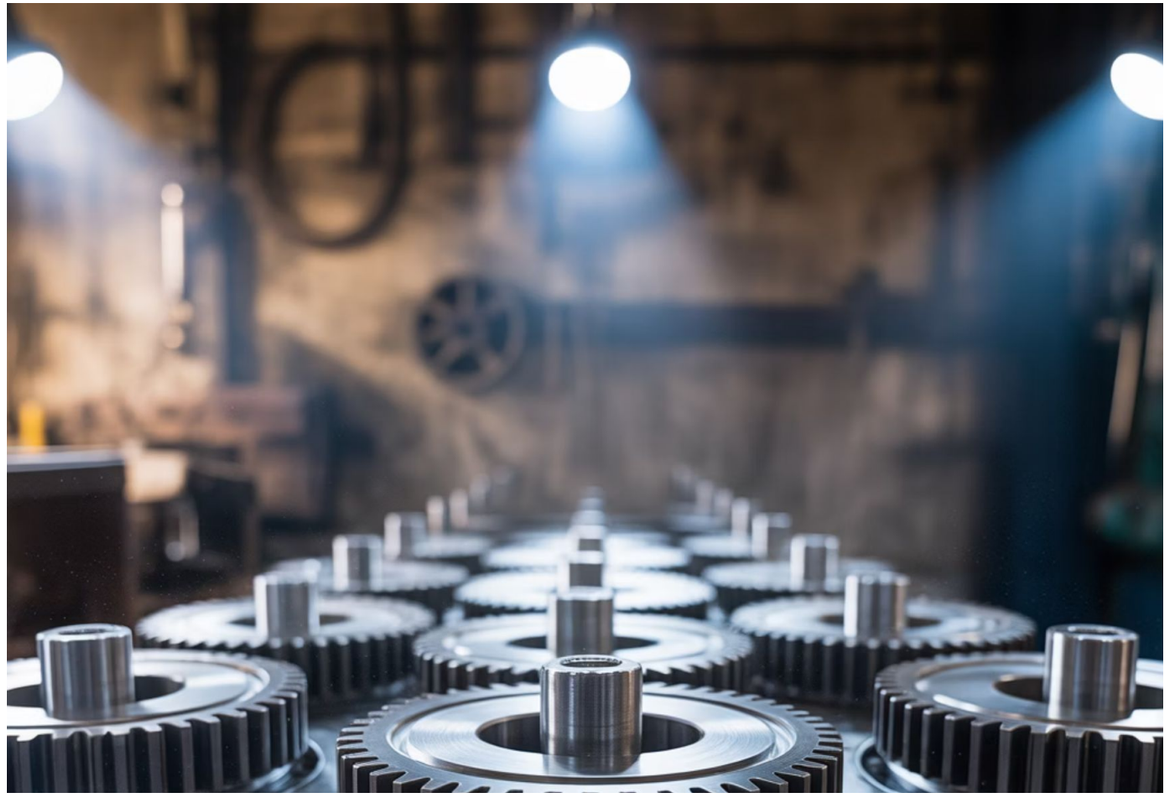
Funciona igual de bien para Java, C#, Python, PHP o cualquier paradigma de programación. La abstracción es su fortaleza principal.

Múltiples aplicaciones

Útil en análisis de requisitos, diseño de arquitectura, documentación técnica y comunicación con clientes y desarrolladores.

Dos grandes familias de diagramas





Diagramas de Estructura

Muestran cómo está **organizado** el sistema

📸 Como una foto fija

- Clases y objetos
- Paquetes y componentes
- Arquitectura estática

Diagramas de Comportamiento

Muestran cómo **actúa** el sistema en el tiempo

🎬 Como una película

- Casos de uso
- Secuencias e interacciones
- Flujos y actividades



Diagramas de Estructura (Parte I)

Diagramas de Clases

El más fundamental en programación orientada a objetos. Define entidades del sistema con sus atributos, métodos y relaciones. Es el blueprint de tu aplicación.

Diagramas de Objetos

Muestran instancias concretas de las clases en un momento específico. Útiles para validar el diseño con datos reales y casos de uso específicos.

Diagramas de Paquetes

Organizan el sistema en módulos lógicos, mostrando dependencias entre diferentes partes del código. Esencial para mantener la modularidad.



Diagramas de Estructura (Parte II)

1

Diagramas de Componentes

Representan módulos ejecutables, APIs, librerías y servicios. Muestran interfaces públicas y dependencias externas del sistema.

2

Diagramas de Despliegue

Mapean la infraestructura física: servidores, dispositivos, bases de datos y redes. Cruciales para la arquitectura de sistemas distribuidos.

3

Estructura Compuesta

Detallan las partes internas de una clase o componente, mostrando cómo interactúan sus elementos internos para lograr funcionalidad compleja.

4

Diagramas de Perfil

Permiten crear estereotipos y extensiones personalizadas de UML para dominios específicos como web, móvil o sistemas embebidos.

Diagramas de Comportamiento (Parte I)

Casos de Uso

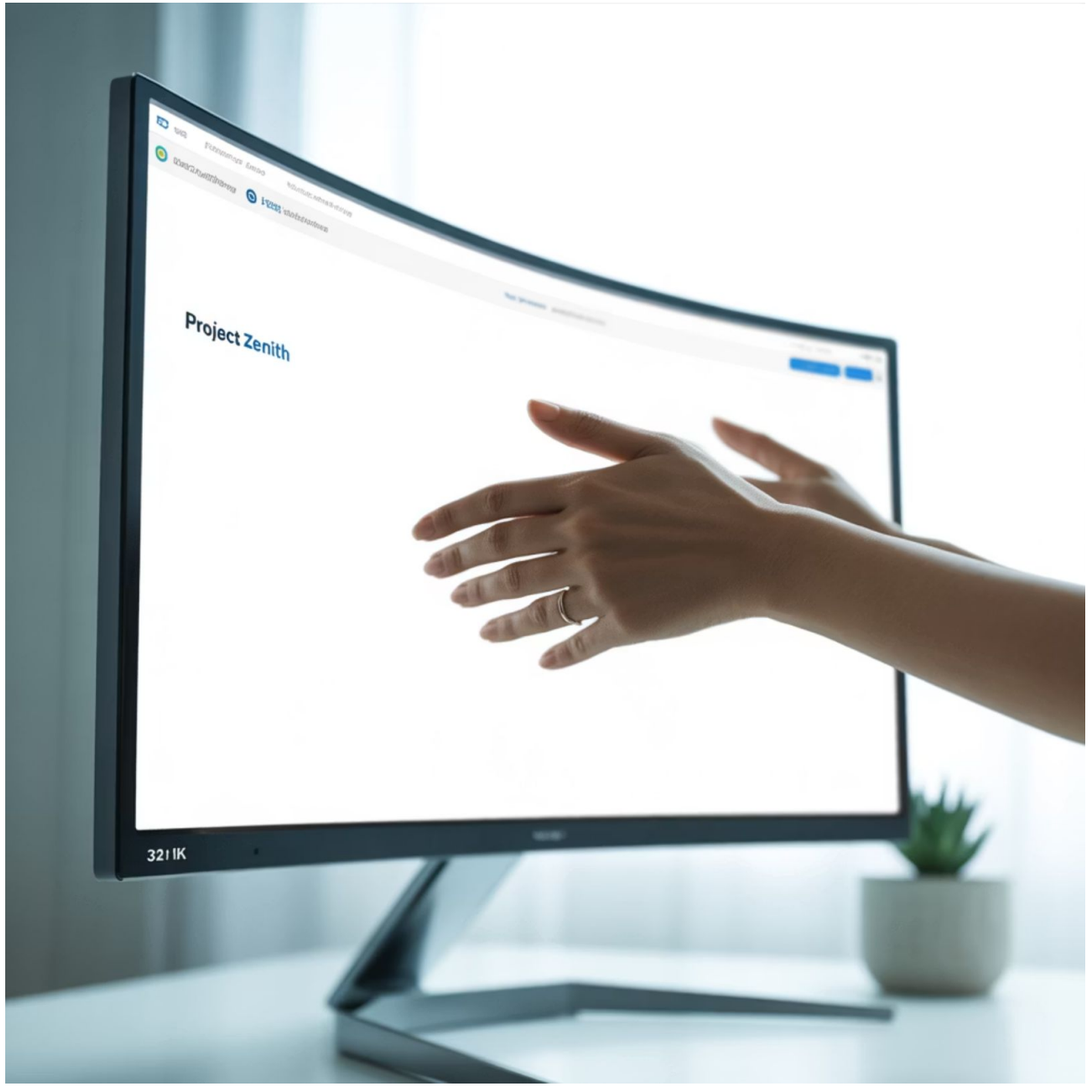
Definen qué hace el sistema desde la perspectiva del usuario. Identifican actores externos y sus objetivos principales.

Diagramas de Actividad

Modelan flujos de trabajo, procesos de negocio y algoritmos complejos. Similares a los diagramas de flujo pero más potentes.

Máquina de Estados

Describen el ciclo de vida de un objeto, mostrando estados posibles y las transiciones que los provocan durante su existencia.





Diagramas de Comportamiento (Parte II)

Diagramas de Secuencia

Muestran mensajes ordenados cronológicamente entre objetos. Perfectos para documentar APIs y protocolos de comunicación.

Diagramas de Comunicación

Enfatizan las colaboraciones y relaciones estructurales entre objetos, complementando la vista temporal de los diagramas de secuencia.

Vista de Interacción

Combinan elementos de actividad y secuencia para proporcionar una visión global de interacciones complejas del sistema.

Diagramas de Temporización

Modelan cambios de estado en función del tiempo real. Esenciales para sistemas críticos y aplicaciones de tiempo real.

¿Cuándo usar cada tipo de diagrama?

Análisis de Requisitos

Usa **Casos de Uso** para capturar funcionalidades desde la perspectiva del usuario y validar el alcance del proyecto con stakeholders.

Modelado de Procesos

Los diagramas de **Actividad** son ideales para workflows, procesos de negocio y algoritmos complejos que requieren decisiones y paralelismo.

Escenarios Específicos

Secuencia y **Comunicación** documentan interacciones detalladas entre objetos, perfectos para APIs y casos de uso complejos.

Diseño Estático

Clases y **Paquetes** definen la estructura del código, organizando entidades y sus relaciones de manera coherente y mantenible.

Arquitectura de Sistema

Componentes y **Despliegue** mapean la infraestructura técnica, desde módulos de software hasta servidores físicos.

Buenas Prácticas en UML

Modela lo necesario

Prioriza claridad sobre detalle exhaustivo. Un diagrama simple y comprensible es más valioso que uno complejo y confuso.

Mantén consistencia

Asegúrate de que todos los diagramas del proyecto usen la misma nomenclatura, convenciones y nivel de abstracción.

Usa lenguaje del dominio

Los nombres de clases, métodos y atributos deben reflejar el vocabulario del negocio, facilitando la comunicación con clientes.

Versiona junto al código

Los diagramas UML deben evolucionar con el código fuente. Mantenlos actualizados y bajo control de versiones.



Herramientas UML para Desarrolladores

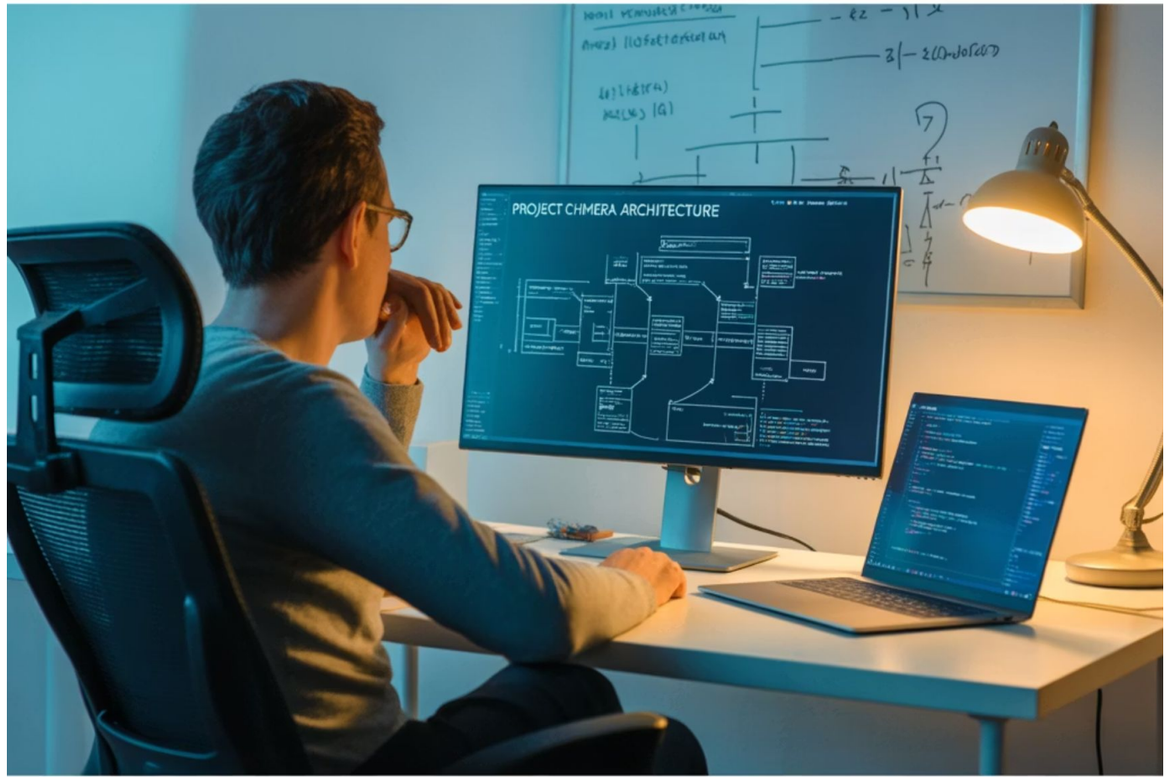
Herramientas basadas en texto

Ideales para integración con control de versiones y automatización:

- **PlantUML** - Sintaxis simple, integración con IDEs
- **Mermaid** - Compatible con GitHub, Markdown

Herramientas gráficas

- **Visual Paradigm** - Profesional, completa
- **StarUML** - Gratuita, multiplataforma
- **draw.io** - Online, colaborativa



En clase empezaremos con PlantUML por su simplicidad y porque permite versionar los diagramas como código fuente.



Actividad Práctica Guiada

Investigación Individual

Cada alumno elige un tipo de diagrama UML y prepara una micro-investigación de 5 puntos clave sobre su uso y aplicación práctica.

Estructura de la Investigación

Preparar exactamente 5 bullets: **para qué sirve, notación básica, pregunta que resuelve, ejemplo práctico, buena práctica**.

Micro-exposición

Presentación individual de 30-45 segundos donde cada estudiante comparte sus hallazgos de manera concisa y clara.

04

Síntesis General

Cierre colaborativo donde conectamos todos los tipos de diagramas y discutimos cómo se complementan en un proyecto real de desarrollo.

¡Manos a la obra! Empecemos a explorar el poder del modelado UML.